

Sesión 201

Inteligencia Artificial con Python

Instituto Politécnico Nacional
Centro de Investigación en Computo

M. Alan Badillo Salas

alan@nomadacode.com

28 de mayo de 2026

Resumen

En esta segunda semana del curso profundizaremos en el uso de las herramientas científicas de Numpy, Pandas, Matplotlib y Seaborn. También estudiaremos a detalle el perceptrón y los métodos de optimización estocástica, para determinar un modelo base para los problemas de regresión y clasificación.

Introducción

Numpy es la librería base para el cómputo científico en Python, con esta librería podemos manipular grandes volúmenes de datos para formar modelos de aprendizaje, esta librería construye un objeto base llamado el *arreglo n -dimensional* o *n -arreglo*, el cual se puede redimensionar en formas complejas para representar matrices y tensores de información numérica. Pandas a su vez, es una librería que mejoran la manipulación de datos tabulares, mediante dos objetos importantes, la *Serie* y el *DataFrame*, los cuales permiten representar un vector de datos (o columna de datos) y una tabla (o conjunto de columnas). Así, mediante estas dos librerías podemos manipular conjuntos de datos y preparar un **cubo de datos** para analizarlo y entrenar modelos de aprendizaje.

En estas notas de la sesión, revisaremos tres puntos principales del *Machine*

Learni (*aprendizaje Automático*), las aplicaciones en la inteligencia artificial y la conexión con las redes neuronales:

1. El *Aprendizaje Automático* y el *Problema del Aprendizaje Supervisado*
2. El *Cubo de Datos* y la separación del entrenamiento y validaciones
3. El Modelo de Regresión Lineal, el *Perceptrón* y la *Neurona Artificial*

Con esto, tendremos las herramientas y visión suficientes para continuar adentrándonos en el mundo de la inteligencia artificial, las redes neuronales, el aprendizaje profundo y los transformadores.

1. El *Aprendizaje Automático* y el *Problema del Aprendizaje Supervisado*

El *Aprendizaje Automático* o *Machine Learning* consiste en establecer modelos de aprendizaje que explotan la información disponible, para construir modelos y espacios numéricos en los que se modelan curvas de predicción y conexiones geométricas, con el fin de resolver problemas de forma automática y consistente, formando así máquinas capaces de tomar decisiones basadas en información estadística, que aproxime y optimice la verdad global y local de los datos.

El *Problema del Aprendizaje Supervisado* es uno de los más importantes dentro de los modelos de aprendizaje y la ciencia de datos, ya que busca construir modelos capaces de predecir la respuesta observada de los datos y así guiar la toma de decisiones automáticas reflejadas directamente de la información disponible. Esto tiene aplicaciones desde la ciencia de datos y estadística, para empresas y gobiernos, pero también en robótica y medicina. El aprendizaje consiste en recopilar datos que puedan ser representados numéricamente, para formar vectores de información, los cuales podríamos pensar como puntos de verdad dentro de un sistema o fenómeno observado. A través de estos datos numéricos y dependiendo si su naturaleza es numérica o categórica, estos revelarán patrones con los que se podría relacionar la respuesta observada en los datos. Así, si tenemos un conjunto de observaciones, las separamos en información que alimenta el modelo y respuestas deducidas por el modelo, tendremos un mecanismo automático que tome la información de entrada, para producir una respuesta como salida. Mediante técnicas de optimización y los parámetros del modelo, podemos ajustar los parámetros para que la respuesta arrojada sea muy similar a la respuesta real observada. Entonces, con

suficientes observaciones, podríamos iterar y optimizar sucesivamente, hasta que el modelo de aprendizaje haya aprendido o capturado lo suficiente, para que las predicciones que arroja tengan poca diferencia con la realidad de los datos. Finalmente, este modelo podrá ser consumido en cualquier momento para predecir respuestas desconocidas para la información que se le alimenta, por ejemplo, saber cuánto cuesta una casa, dados sus características como el número de baños, metros cuadrados del terreno o calidad de la cocina. A todo esto se le conoce como el problema del aprendizaje supervisado: Como descubrir el comportamiento de una respuesta que se ha observado.

Para poner en contexto este problema usemos numpy para construir datos sintéticos que modelen una respuesta cruzada entre dos variables, agregando una cantidad de ruido para ver cómo se comporta la regresión lineal.

1.1. Ejemplo 1 - Predecir el nivel de estrés de un trabajador

El nivel de estrés de un trabajador se podría modelar como una respuesta que depende de la carga de trabajo y el salario que gana. El punto de equilibrio podría ser tener una carga de trabajo intensa de 10 horas a la semana y un salario promedio de 15,000, a partir de ahí un extremo podría ser tener 2 veces más horas de trabajo para estresarse al mismo ritmo o ganar el doble para reducir el estrés a la mitad, entonces, el modelo podría quedar como:

$$estres = 1 + \frac{carga}{10} - \frac{salario}{15,000} \quad (1)$$

Además si pensamos que algunos trabajadores tienen resistencia hacia altas cargas de trabajo, por ejemplo, pueden trabajar 3 horas más con el mismo efecto de estrés, aunque otros sufren un efecto inverso que con 3 horas menos generan más estrés que el resto, y lo mismo con el salario, quizás ganar 5 mil más genera menos estrés, pero para otro es al revés y con esos 5 mil generan más estrés, por lo que podríamos agregar un factor aleatorio de resistencia o aumento al estrés en la carga de trabajo y otro en el salario.

$$estres = 1 + \frac{carga - resistencia_{carga}}{10} - \frac{salario - resistencia_{salario}}{15,000} \quad (2)$$

$$resistencia_{carga} \sim \mathbf{Normal}(0, 3)$$

$$resistencia_{salario} \sim \mathbf{Normal}(0, 5,000)$$

Si la resistencia es positiva, significa que el nivel de estrés bajará, pero si es negativa, entonces aumentará.

Reorganizando las ecuaciones en la forma normal de la regresión lineal tenemos:

$$\begin{aligned}\hat{y} &= 1 + (x_1 - \delta_{x_1})\frac{1}{10} - (x_2 - \delta_{x_2})\frac{1}{15,000} \\ \hat{y} &\leftarrow \textit{estres} \\ x_1 &\leftarrow \textit{carga} \\ x_2 &\leftarrow \textit{salarario} \\ \delta x_1 &\leftarrow \textit{efecto}_{\textit{carga}} \\ \delta x_2 &\leftarrow \textit{efecto}_{\textit{salarario}}\end{aligned}\tag{3}$$

Y considerando los parámetros desconocidos $\textit{bias}$, β_1 y β_2 , podemos crear una simulación de datos aleatorios que consideren un efecto aleatorio para las cargas y salarios.

$$\begin{aligned}\hat{y} &= \textit{bias} + (x_1 - \delta_{x_1})\beta_1 + (x_2 - \delta_{x_2})\beta_2 \\ \textit{bias} &= 1 \\ \beta_1 &= \frac{1}{10} \\ \beta_2 &= \frac{-1}{15,000}\end{aligned}\tag{4}$$

Usaremos Numpy y Pandas para simular $n = 1,000$ datos aleatorios de trabajadores con cargas aleatorias, más un efecto aleatorio, salarios aleatorios, más un efecto aleatorio y el estrés resultante.

```

1 import numpy
2 import pandas
3
4 from matplotlib import pyplot
5 import seaborn
6
7 def estres_empleado(carga, salario):
8     efecto_carga = numpy.random.normal(0, 2)
9     efecto_salario = numpy.random.normal(0, 3_000)
10    efecto_bias = numpy.random.normal(0, (abs(
        efecto_carga) / 10 + abs(efecto_salario) / 15_000
    ) / 2)
11
12    return (
13        (1 - efecto_bias) +
14        (carga - efecto_carga) / 10 -
15        (salario - efecto_salario) / 15_000
16    )

```

```

17
18 for i in range(10):
19     carga = int(numpy.random.normal(10, 1))
20     salario = numpy.round(numpy.random.normal(15_000, 1
21         _000), 2)
22     estres = estres_empleado(carga, salario)
23     print(
24         f"Empleado {i + 1:2d}: "
25         f"{carga:2d}h "
26         f"${salario:9.2f} "
27         f"{estres:4.2f}"
28     )

```

Con la función de estrés del empleado, podemos modelar un factor de estrés con efectos aleatorios dada la carga laboral, el salario y un efecto aleatorio.

```

Empleado  1:  7h $ 15032.61 0.21
Empleado  2:  8h $ 16216.81 0.33
Empleado  3:  9h $ 15034.38 1.01
Empleado  4:  9h $ 15329.17 0.92
Empleado  5: 10h $ 15456.23 0.99
Empleado  6:  9h $ 12605.80 0.50
Empleado  7:  8h $ 13933.96 0.88
Empleado  8: 10h $ 16328.66 0.99
Empleado  9: 10h $ 14731.78 0.55
Empleado 10: 10h $ 14530.23 1.29

```

Observamos que cuando la carga laboral es inferior a las 10 horas el estrés se reduce, y cuando el salario supera los \$15,000 también.

Entonces, para $n = 1,000$ simulaciones de empleados aleatorios, con una carga laboral cercana a las 10 horas, con una desviación de 3 horas (de 7 a 13 horas normales en 67% de los empleados) y también un salario cercano a los \$15,000 con una desviación de \$5,000 (de \$5,000 a \$5,000 pesos mensuales en 67% de los empleados), entonces, el estrés podría ser deducido por la función de estrés del empleado a aplicada a cada carga laboral y salario.

```

1 n = 1_000
2
3 cargas = numpy.random.normal(10, 3, n).astype(int)
4 salarios = numpy.random.normal(15_000, 5_000, n).round
5     (2)
6 estreses = pandas.Series(zip(cargas, salarios)).map(
7     lambda t: estres_empleado(t[0], t[1])).round(2)

```

```

6
7 pandas.DataFrame({
8     "CARGA": cargas,
9     "SALARIO": salarios,
10    "ESTRES": estreses,
11 })

```

Teniendo como resultado un conjunto de datos de cargas laborales, salarios y estrés para cada observación simulada.

	CARGA	SALARIO	ESTRES
0	10	18205.19	0.77
1	7	16133.27	0.13
2	8	15754.10	0.86
3	14	12583.51	2.05
4	9	14025.50	1.06
...	...	...	...
995	5	14347.02	0.48
996	13	21470.59	1.14
997	11	11801.91	1.04
998	14	21902.42	0.10
999	10	12737.74	1.16

1000 rows x 3 columns

Si cruzamos los datos y discretizamos los salarios por cuartiles, podemos generar un mapa de calor que explique el estrés por carga laboral y salario, concentrando los datos primero por salario (**Figura 1**).

```

1 cruce = pandas.DataFrame({
2     "CARGA": cargas,
3     "SALARIO": pandas.qcut(
4         salarios, q=[0, 0.25, 0.5, 0.75, 1],
5         labels=(
6             "SALARIO-BAJO", "SALARIO-MEDIO-BAJO",
7             "SALARIO-MEDIO-ALTO", "SALARIO-ALTO"
8         )
9     ),
10    "ESTRES": estreses,
11 }).groupby(
12     ["CARGA", "SALARIO"],
13     observed=False
14 ).mean().unstack()
15

```

```

16 figure, axis = pyplot.subplots(1, 1, figsize=(3, 10))
17
18 seaborn.heatmap(
19     cruce.divide(cruce.mean(axis=1).abs() + 1, axis=0),
20     cmap="coolwarm",
21     ax=axis
22 )
23
24 axis.set_title(
25     "Mapa de concentracion del estres en trabajadores\n"
26     "Por nivel salarial\n"
27 )
28
29 pyplot.xlabel("")
30 pyplot.ylabel(
31     "Horas intensas de carga laboral a la semana"
32 )
33
34 pyplot.savefig("fig1.png", dpi=300, bbox_inches="tight")
35
36 pyplot.show()

```

De forma compacta podemos visualizar el promedio de las concentraciones de estrés por salario (**Figura 2**).

```

1 figure, axis = pyplot.subplots(1, 1, figsize=(5, 1))
2
3 seaborn.heatmap(
4     cruce.mean(axis=0).unstack(),
5     cmap="coolwarm",
6     ax=axis
7 )
8
9 axis.set_title(
10     "Concentracion promedio del estres "
11     "en trabajadores\n"
12     "Por nivel salarial\n"
13 )
14
15 pyplot.xlabel("")
16
17 pyplot.savefig("fig2.png", dpi=300, bbox_inches="tight")
18
19 pyplot.show()

```

Mapa de concentración del estrés en trabajadores
Por nivel salarial

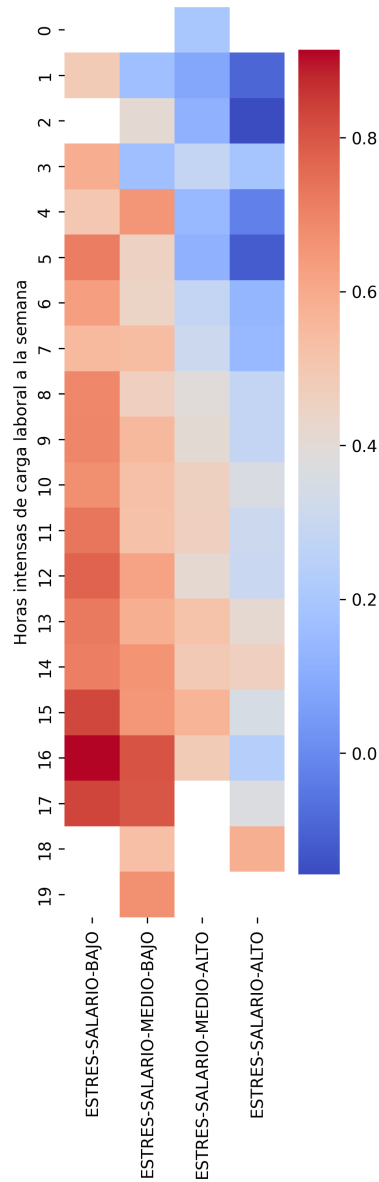


Figura 1: Visualización de la concentración de estrés por salario

Aquí observamos que el mayor estrés promedio se ubica debajo del primer cuartil de los salarios (inferiores a \$11,770) y el menor estrés arriba del tercer cuartil (superiores a \$18,270).

De forma similar podemos explicar la concentración del estrés explicada por

Concentración promedio del estrés en trabajadores
Por nivel salarial

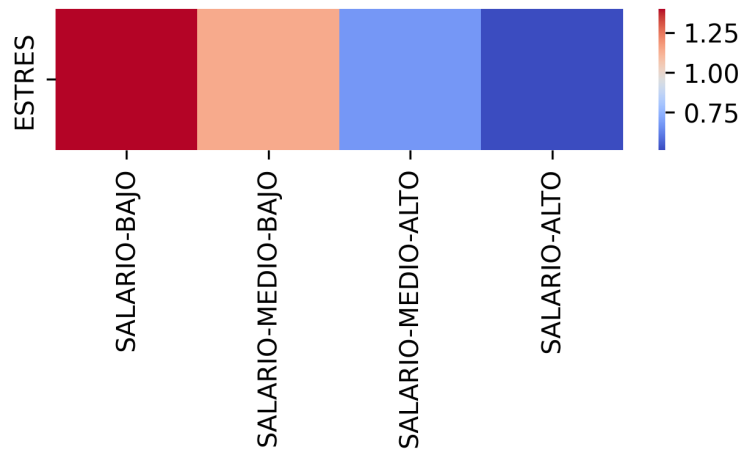


Figura 2: Visualización compacta del estrés por salario

carga laboral (**Figura 3** y **Figura 4**).

```

1 cruce = pandas.DataFrame({
2     "CARGA": cargas,
3     "SALARIO": pandas.qcut(
4         salarios, q=[0, 0.25, 0.5, 0.75, 1],
5         labels=(
6             "SALARIO-BAJO", "SALARIO-MEDIO-BAJO",
7             "SALARIO-MEDIO-ALTO", "SALARIO-ALTO"
8         )
9     ),
10    "ESTRES": estreses,
11 }).groupby(
12     ["SALARIO", "CARGA"],
13     observed=False
14 ).mean().unstack()
15
16 figure, axis = pyplot.subplots(1, 1, figsize=(10, 3))
17
18 seaborn.heatmap(
19     cruce.divide(cruce.mean(axis=1).abs() + 1, axis=0),
20     cmap="coolwarm",
21     ax=axis
22 )

```

```

23
24 axis.set_title(
25     "Mapa de concentracion del estres en trabajadores\n"
26     "Por carga laboral\n"
27 )
28
29 pyplot.xlabel(
30     "Horas intensas de carga laboral "
31     "a la semana"
32 )
33 pyplot.ylabel("")
34
35 pyplot.savefig("fig3.png", dpi=300, bbox_inches="tight")
36
37 pyplot.show()

```

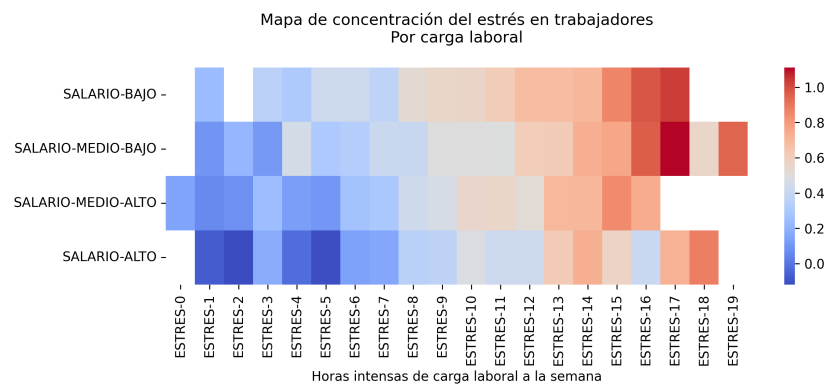


Figura 3: Visualización de la concentración de estrés por carga laboral

```

1 figure, axis = pyplot.subplots(1, 1, figsize=(10, 1))
2
3 seaborn.heatmap(
4     cruce.mean(axis=0).unstack(),
5     cmap="coolwarm",
6     ax=axis
7 )
8
9 axis.set_title(
10     "Concentracion promedio del estres "
11     "en trabajadores\n"
12     "Por nivel salarial\n"
13 )

```

```

14
15 pyplot.xlabel(
16     "Horas intensas de carga laboral "
17     "a la semana"
18 )
19
20 pyplot.savefig("fig4.png", dpi=300, bbox_inches="tight")
21
22 pyplot.show()

```

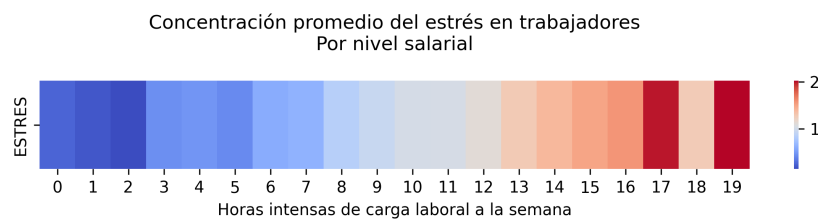


Figura 4: Visualización compacta del estrés por carga laboral

Aquí observamos que el estrés promedio aumenta cuando la carga laboral aumenta.

Con estas visualizaciones podemos concluir que el estrés promedio disminuye de forma inversa al salario y de forma directa a la carga de trabajo. Esto significa que hay bastantes argumentos para determinar que el estrés se comporta de forma lineal respecto a la carga laboral y salario del empleado.

Más aún, podemos ver estas correlaciones de forma explícita visualizando el patrón lineal que forman las observaciones de estrés respecto a la carga laboral y el salario (**Figura 5** y **Figura 6**).

```

1 figure, axis = pyplot.subplots(1, 1, figsize=(10, 10))
2
3 seaborn.scatterplot(
4     x=cargas,
5     y=estreses,
6     color="indigo",
7     alpha=0.3,
8     ax=axis
9 )
10
11 axis.set_title(
12     "Estres por carga laboral\n"

```

```

13 )
14
15 pyplot.xlabel(
16     "Horas intensas de carga laboral "
17     "a la semana"
18 )
19 pyplot.ylabel("Estrés")
20
21 pyplot.savefig("fig5.png", dpi=300, bbox_inches="tight")
22
23 pyplot.show()

```

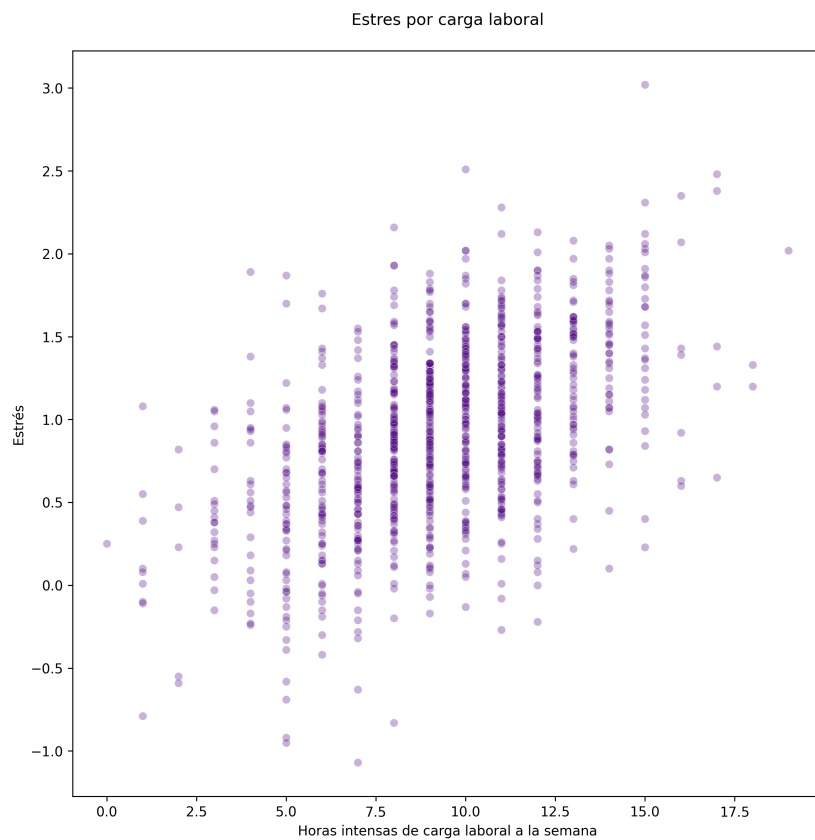


Figura 5: Correlación entre el estrés y la carga laboral

```

1 figure, axis = pyplot.subplots(1, 1, figsize=(10, 10))
2
3 seaborn.scatterplot(
4     x=salarios,

```

```

5     y=estreses,
6     color="orange",
7     alpha=0.3,
8     ax=ax
9 )
10
11 axis.set_title(
12     "Estres por salario\n"
13 )
14
15 pyplot.xlabel("Salario")
16 pyplot.ylabel("Estres")
17
18 pyplot.savefig("fig6.png", dpi=300, bbox_inches="tight")
19
20 pyplot.show()

```

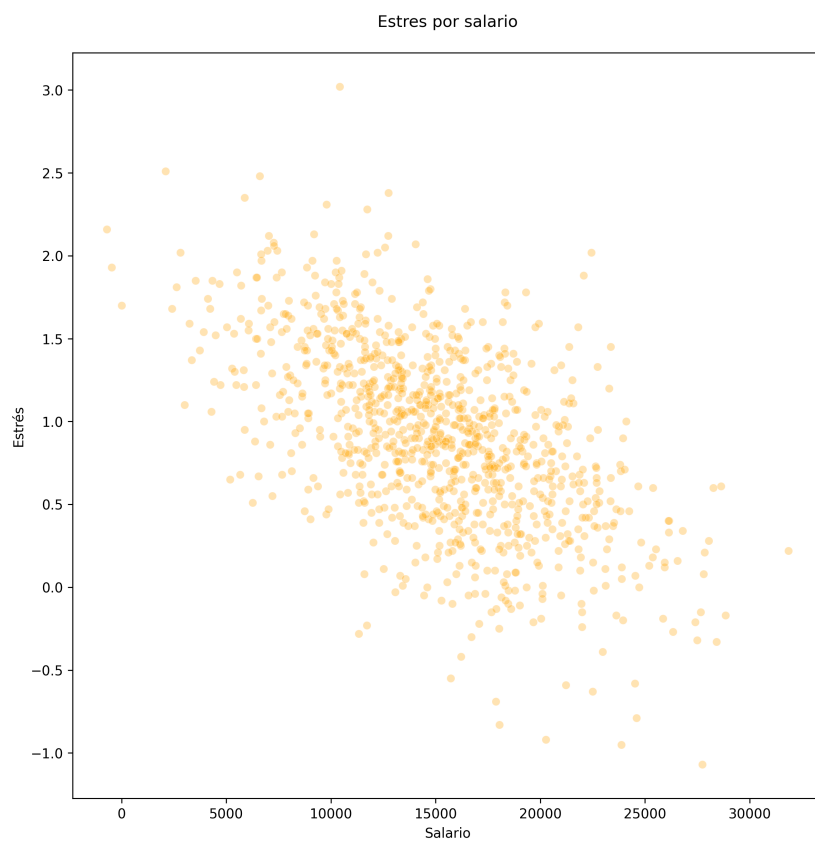


Figura 6: Correlación entre el estrés y el salario

Ahora es más fácil ver la correlación directa entre el estrés y la carga laboral y la correlación inversa entre el estrés y el salario. Además, nota que como la carga laboral es entera (medida en horas exactas), la gráfica parece alineada en franjas, esto es normal en datos discretos.

Finalmente, podemos visualizar el espacio donde viven las cargas laborales y salarios, aunque no explicarían más allá de donde se concentran los empleados, y como son datos sintéticos simulados, esto parecerá una nube de puntos uniforme, haría falta determinar cómo se comporta la respuesta (el estrés) bajo esta nube de puntos (**Figura 7**).

```
1 figure, axis = pyplot.subplots(1, 1, figsize=(10, 10))
2
3 seaborn.scatterplot(
4     x=cargas,
5     y=salarios,
6     color="gray",
7     alpha=0.3,
8     ax=axis
9 )
10
11 axis.set_title(
12     "Distribucion de las cargas laborales y salarios\n"
13 )
14
15 pyplot.xlabel(
16     "Horas intensas de carga laboral "
17     "a la semana"
18 )
19 pyplot.ylabel("Salario")
20
21 pyplot.savefig("fig7.png", dpi=300, bbox_inches="tight")
22
23 pyplot.show()
```

Observa que la nube de puntos se concentra cerca de las 10 horas de carga laboral y un salario cercano a \$15,000. Los puntos periféricos, representarán puntos atípicos que podrían ser lecturas erróneas, como salarios negativos u horas extremadamente altas o incluso negativas.

Para que la nube de puntos tenga sentido, dado que solo estamos representando la concentración de la carga laboral y el salario (solo las características o información de entrada del empleado), faltaría colorear la respuesta (el factor

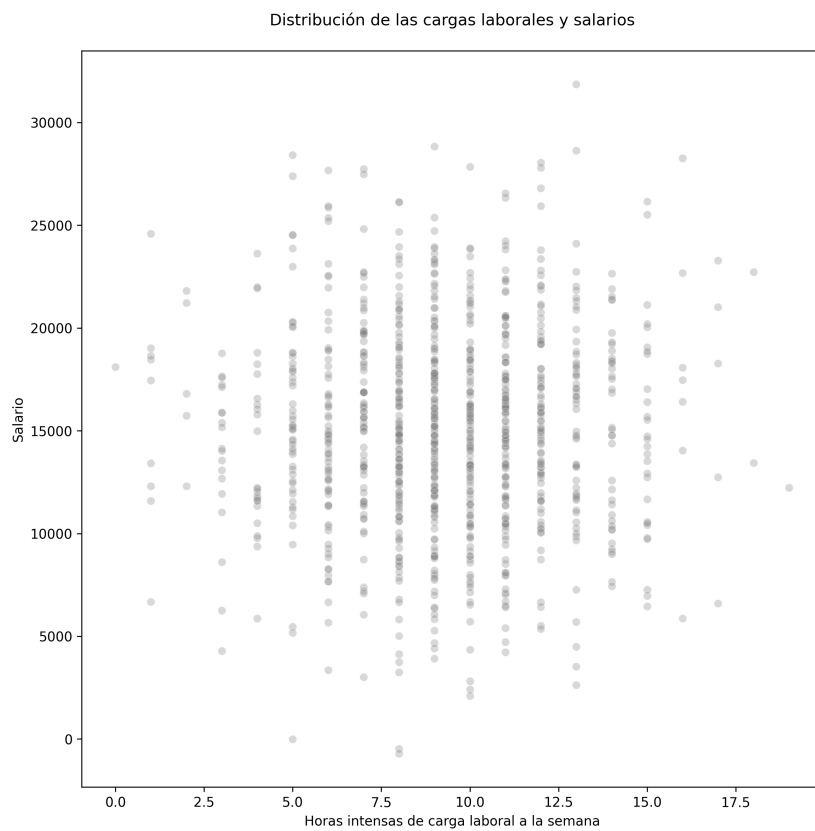


Figura 7: Espacio de cargas laborales y salarios

de estrés) formando un contraste (**Figura 8**).

```

1 figure, axis = pyplot.subplots(1, 1, figsize=(10, 10))
2
3 seaborn.scatterplot(
4     x=cargas,
5     y=salarios,
6     hue=estreses,
7     alpha=0.8,
8     ax=axis
9 )
10
11 axis.set_title(
12     "Distribucion de las cargas laborales y salarios\n"
13     "Con estres continuo\n",
14 )
15

```

```

16 axis.legend(title="Factor de estrés")
17
18 pyplot.xlabel(
19     "Horas intensas de carga laboral "
20     "a la semana"
21 )
22 pyplot.ylabel("Salario")
23
24 pyplot.savefig("fig8.png", dpi=300, bbox_inches="tight")
25
26 pyplot.show()

```

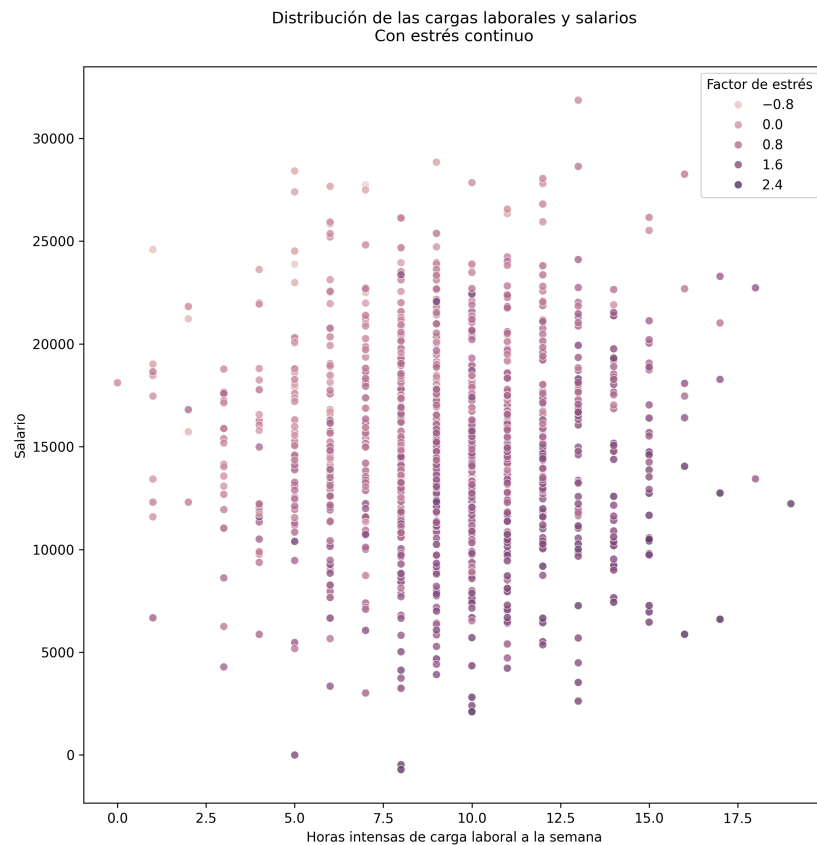


Figura 8: Espacio de cargas laborales y salarios contrastado por estrés

Y mejor aún, podemos forzar el estrés para que explique si es o no mayor o igual a 1, esto nos permitirá saber además si es posible determinar si el empleado sufrirá un factor de estrés superior o inferior al común, abriendo la posibilidad de transformar la regresión en clasificación (**Figura 9**).


```

1 figure, axis = pyplot.subplots(1, 1, figsize=(10, 10))
2
3 seaborn.scatterplot(
4     x=cargas,
5     y=salarios,
6     hue=estreses > 1,
7     alpha=0.8,
8     ax=axis
9 )
10
11 axis.set_title(
12     "Distribucion de las cargas laborales y salarios\n"
13     "Con estres mayor a 1\n",
14 )
15
16 axis.legend(title="Factor de estres (mayor a 1)")
17
18 pyplot.xlabel(
19     "Horas intensas de carga laboral "
20     "a la semana"
21 )
22 pyplot.ylabel("Salario")
23
24 pyplot.savefig("fig9.png", dpi=300, bbox_inches="tight")
25
26 pyplot.show()

```

Esta última visualización nos permite interpretar el nivel de separabilidad de la respuesta, para concluir que es un problema altamente separable, dado que el factor de estrés mayor o igual a 1, queda cargado hacia la esquina inferior derecha (una alta carga laboral y un salario bajo), mientras que el factor de estrés que no supera al 1, queda cargado hacia la esquina superior izquierda (una baja carga laboral y un salario alto).

Ahora, con esta información podemos crear un modelo de aprendizaje que tome las características de los empleados (x_1 y x_2 que representa la carga laboral y el salario), para poder predecir la respuesta observada (y que representa el factor de estrés).

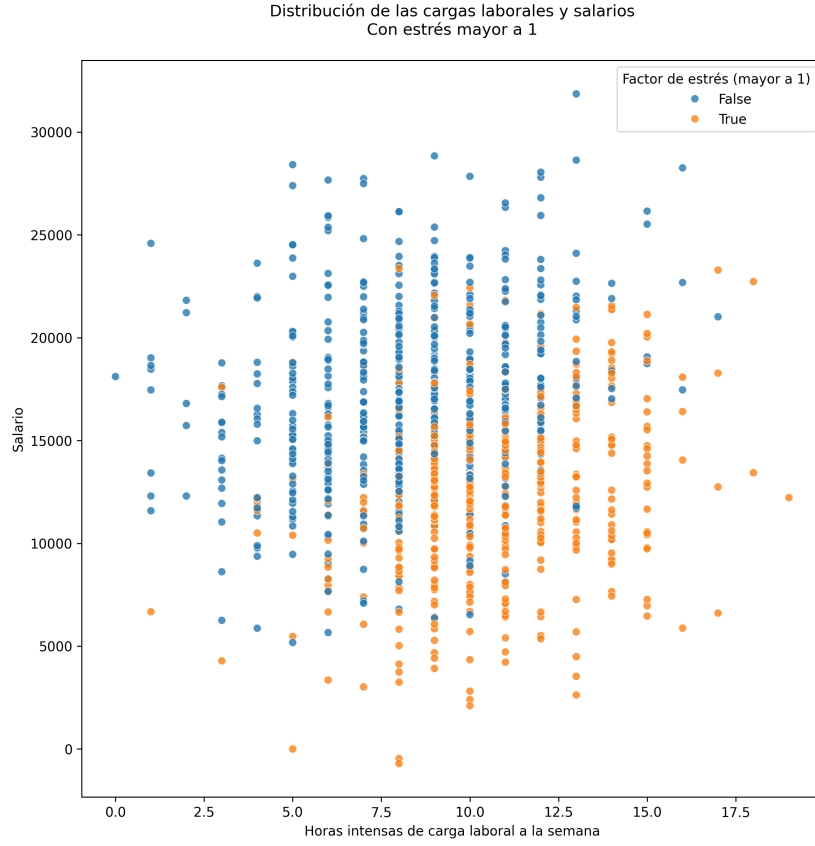


Figura 9: Espacio de cargas laborales y salarios contrastado por estrés mayor o igual a 1

Usando el *Modelo de Regresión Lineal Múltiple*, tenemos:

$$\begin{aligned}\hat{y} &= \Phi(x_1, x_2) \\ \hat{y} &= \phi(x_1, x_2; bias, \beta_1, \beta_2) \\ \hat{y} &= bias + x_1\beta_1 + x_2\beta_2\end{aligned}\tag{5}$$

donde el *bias* representa fácilmente un valor β_0 que nos permitirá expresar de forma más compacta y consistente el modelo de predicción como:

$$\hat{y} = X \cdot \beta\tag{6}$$

donde X será la matriz de información extendida (con una columna de 1's para ajustar el bias), también conocida como la matriz de diseño. Y el vector $\beta = (\beta_0, \beta_1, \beta_2)$ serán los parámetros desconocidos, constantes del modelo ajustado o coeficientes de regresión.

Entonces, nuestro objetivo será ajustar y encontrar los β^* óptimos, para los cuales, la respuesta observada es similar o muy cercana (lo mejor posible) a la respuesta de predicción que produce el modelo:

$$y \approx \hat{y} = X \cdot \beta \quad (7)$$

Considerando que esto deberá ocurrir para cada observación y_i , dado que la respuesta observada no es continua, sino n valores que fueron observados de n empleados (simulados o reales), pero que conforman el conjunto de datos.

$$y_i \approx \hat{y}(x_1^{(i)}, x_2^{(i)}) = X_i \cdot \beta \quad (8)$$

O de forma explícita:

$$y_i \approx \hat{y}(x_1^{(i)}, x_2^{(i)}) = bias^* + x_1^{(i)}\beta_1^* + x_2^{(i)}\beta_2^* \quad (9)$$

Léase, "*La i -ésima observación de la respuesta observada, se aproxima a la respuesta de predicción producida por el modelo*", o en otras palabras, el factor de estrés observado para los empleados, deberá ser próxima al factor de estrés predicho por el modelo que toma la carga laboral multiplicado por una constante óptima β_1^* , el salario multiplicado por una constante óptima β_2^* y una constante óptima $bias^*$, los cuales fueron ajustados mediante optimización o métodos deterministas, y representan valores cercanos al modelo ideal con el que fueron generados los datos.

2. El *Cubo de Datos* y la separación del entrenamiento y validaciones

Ahora pensemos de forma más general, dejando a un lado el origen del conjunto de datos, es decir, si un conjunto de datos se generó de forma sintética simulando observaciones que tienen ruidos aleatorios y un modelo base que produce n observaciones o se realizaron n experimentos y se recopiló la información de eventos reales.

Pero, en general podríamos pensar que sintético o no, tenemos un conjunto de datos observados, que contienen características de un fenómeno observado, que en el *Problema del Aprendizaje Supervisado*, separaría la información de entrada (características inherentes que podríamos conocer sobre el fenómeno al realizar una observación) y la respuesta (variable a la que se le desea aplicar una observación).

Entonces, podemos formar una matriz de características $X = [x_1, x_2, \dots, x_k]$

que concentre la información de los vectores $x_1, x_2, \dots, x_k$, cada uno de tamaño n , con las n observaciones para cada característica del conjunto de datos. Por lo tanto, X tendrá dimensión de $n \times k$ y le llamaremos *El Cubo de Datos* o *cubo de información*.

En los modelos de *Machine Learning* (*Aprendizaje Automático*), usaremos *cubos de datos* numéricos, para establecer modelos matemáticos puros, que sean capaces de ajustar modelos capaces de analizar de forma automática los datos, usando técnicas geométricas, estadísticas y de inteligencia artificial (como las redes neuronales o los *transformers*).

Ahora, teniendo un cubo de datos, es importante separar las n observaciones en dos conjuntos:

1. **Conjunto de entrenamiento** - Toma un gran porcentaje de las observaciones para entrenar el *modelo de aprendizaje*
2. **Conjunto de pruebas** - Toma un pequeño porcentaje de las observaciones (el restante) para validar el *modelo de aprendizaje*

Con esto seremos capaces de determinar de una forma más realista que tan bien se adapta el modelo de aprendizaje a observaciones desconocidas (las observaciones que reservamos en las pruebas y de las cuales el modelo no tiene conocimiento).

Esta estrategia permite determinar cómo se comportará el modelo en observaciones que de las que no tiene conocimiento ni de sus características, ni de su respuesta. Si el modelo logra buenas predicciones en este conjunto de pruebas o validación, podríamos esperar que con otros datos desconocidos brinde buenas proyecciones.

Si no lo hacemos así, corremos el riesgo de que el modelo aprenda demasiado sobre las observaciones completas y no sepamos si para otros datos desconocidos dará buenos resultados, es decir, no tendríamos más datos para determinar si lo está haciendo bien o se sobreajustó a los datos que aprendió.

El problema de sobreajuste significa que el modelo podría ser muy bueno prediciendo respuestas que aprendió en su ajuste, pero para otras observaciones (por ejemplo, en el conjunto de pruebas) las predicciones son malas. Si la predicción es buena en el conjunto de pruebas, nos podríamos fiar de que el modelo sea bueno prediciendo valores más allá de lo que aprendió. Pero hay que cuidar que las observaciones sean suficientes para cubrir grandes espacios y no cometer el error de enseñarle solo una región local o sesgada

del fenómeno o evento, por ejemplo, el estrés en empleados de una empresa y luego querer predecir el estrés en otra empresa, los factores de estrés podrían estar sesgados, si solo recopilamos datos de cómo se estresan los empleados en una empresa y pretender que las cargas de trabajo y salarios se comportan igual en otras empresas.

Otro detalle importante es aceptar que partir los el cubo de datos en datos de entrenamiento y datos de validación es lo mínimo necesario para mostrar optimismo en la predicción, pero existen estrategias más avanzadas como la *Validación Cruzada* o validaciones avanzadas que permitan concentrar la misma riqueza de diversidad y balance de la respuesta en submuestras para ir evaluando el comportamiento del modelo en diferentes subconjuntos y luego entender el aprendizaje promedio alcanzado y las fluctuaciones o desviaciones entre los diferentes conjuntos.

En resumen podemos pensar que el flujo a seguir para el problema de aprendizaje supervisado se podría ver como:

1. Crear un cubo de datos X y la respuesta y (si hay más respuestas ignorarlas de momento y centrarse solo en una, luego repetir otros modelos para las otras respuestas, aunque esto se puede resolver en las redes neuronales multi-respuesta).
2. Separar el cubo de datos X en X_{train} y X_{test} para entrenar el modelo de aprendizaje solo con X_{train} y luego validar con X_{test} .
3. Calcular las predicciones sobre X_{test} y calcular las métricas de error o pérdida según el problema (por ejemplo, MSE , $RMSE$ y MAE para regresión o BCE , *Matriz de Confusión*, AUC , ROC para clasificación binaria).

2.1. Ejemplo 2 - Adquirir y separar un cubo de datos

Para construir el cubo de datos, cargaremos primero el conjunto de datos, por ejemplo, del archivo *estres.csv*.

```
1 import numpy
2 import pandas
3
4 from matplotlib import pyplot
5 import seaborn
6
7 estres_empleados = pandas.read_csv(
```

```

8         ".../conjuntos/estres.csv"
9     )
10
11     estres_empleados

```

La construcción del cubo de datos X quedará como:

```

1 X = estres_empleados[["CARGA", "SALARIO"]].values

```

Y la construcción del vector de respuesta y quedará como:

```

1 y = estres_empleados["ESTRES"]

```

Ahora tenemos una matriz X de características informativas y un vector de respuesta y :

```

1 print("X", X.shape)
2 print("y", y.shape)

```

Con dimensiones $X \leftarrow (1,000 \times 2)$ y $y \leftarrow (1,000 \times 1)$.

Ahora podemos partir el los datos en entrenamiento y pruebas:

```

1 from sklearn.model_selection import train_test_split
2
3 X_train, X_test, y_train, y_test = train_test_split(
4     X, y,
5     train_size=800
6 )
7
8 print("X_train", X_train.shape)
9 print("X_test", X_test.shape)
10 print("y_train", y_train.shape)
11 print("y_test", y_test.shape)

```

Observa que las dimensiones quedarán como $X_{train} \leftarrow (800 \times 2)$ y $y_{train} \leftarrow (800 \times 1)$ y $X_{test} \leftarrow (200 \times 2)$ y $y_{test} \leftarrow (200 \times 1)$.

3. El Modelo de Regresión Lineal, el *Perceptrón* y la *Neurona Artificial*

Ahora ya podemos establecer la matriz de diseño X_d que combinará las características de X_{train} con un vector de **1** para el *bias*.

Se utiliza X_{train} y no X para ajustar el modelo de regresión lineal solo a

los datos de entrenamiento y que los datos de prueba X_{test} puedan validar qué tan bien predice el modelo de regresión lineal, reservando estos datos de prueba fuera del modelo.

Entonces el modelo de regresión lineal:

$$\hat{y}_{train} = X_d \cdot \beta \quad (10)$$

donde $\beta = (bias, \beta_1, \beta_2)$ dado que solo hay dos características en X y el bias se incorpora a la matriz de diseño X_d .

```

1 Xd = numpy.array([
2     numpy.zeros_like(X_train[:, 0]) + 1,
3     X_train[:, 0],
4     X_train[:, 1],
5 ]) .T

```

Usando las ecuaciones normales, podemos despejar β asumiendo que baja multicolinealidad y un determinante que no sea cercano a 0 para $\det(X_d^\top X)$:

$$\begin{aligned}
 y &= X\beta \\
 X\beta &= y \\
 X_d^\top X\beta &= X_d^\top y \\
 (X_d^\top X)\beta &= X_d^\top y \\
 \beta &= (X_d^\top X)^{-1} X_d^\top y
 \end{aligned} \quad (11)$$

```

1 betas = numpy.linalg.inv(
2     Xd.T.dot(Xd)
3 ).dot(
4     Xd.T.dot(y_train)
5 )

```

Con esto podemos observar que las β alcanzadas son:

```

1 bias = betas[0]
2 b1 = betas[1]
3 b2 = betas[2]
4
5 print(f"bias = {bias}")
6 print(f"b1 = {b1}")
7 print(f"b2 = {b2}")

```

```

bias = 1.0071789418581023
b1    = 0.0969904966716586
b2    = -6.582394779824864e-05

```

Esto significa que $bias \approx 1,0072$, $\beta_1 \approx 0,097$ y $\beta_2 \approx -6,5874 \cdot 10^{-5}$. Lo que significaría que por cada hora aumentada a la carga el estrés aumenta casi 0,1 y por cada \$10,000 pesos de salario el estrés disminuye 0,6.

Estos resultados son consistentes con nuestro modelo original donde cada 10 horas el estrés aumenta 1 y cada \$15,000 el estrés disminuye 1, además de un valor constante o *bias* de 1 de estrés por defecto.

Ahora podemos observar las visualizaciones más importantes para este modelo de 2 dimensiones. En la **Figura 10** observamos en un ángulo fijo de la nube de datos formada por las dimensiones de $X = (x_1, x_2)$ como base, que representan la carga en horas y el salario y la respuesta y como altura que es el factor de estrés. Observa que en color morado se colorea el estrés más

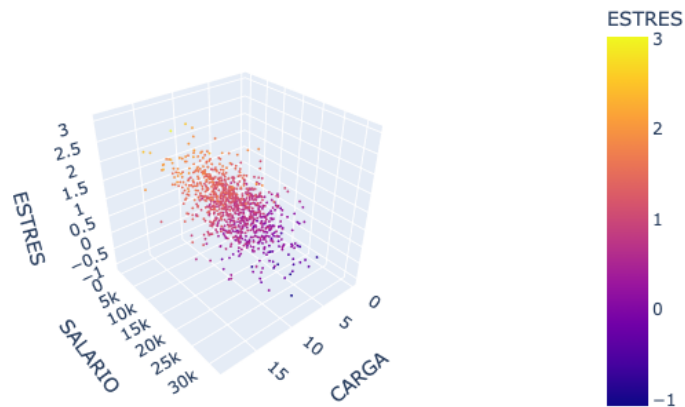


Figura 10: Nube de puntos del estrés dada la carga laboral y el salario

pequeño y en amarillo el más grande. Esto sugiere que a mayor salario habrá un menor estrés.

En la **Figura 11** observamos como el aumento del salario hace que disminuya el estrés. Y en la **Figura 12** observamos como el aumento de la carga laboral hace que incremente también el estrés. Además ver los puntos en forma de franja sugiere que la carga laboral es discreta (no hay cargas laborales intermedias por eso está vacío).

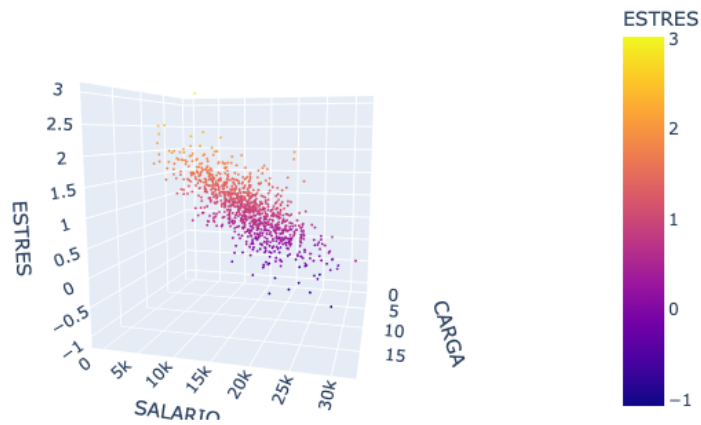


Figura 11: Nube de puntos del estrés proyectando el salario

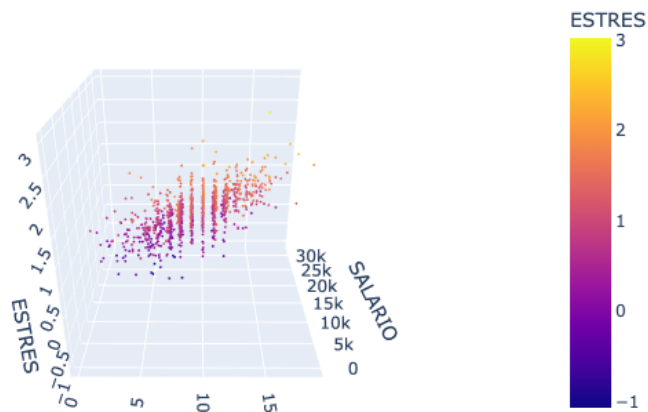


Figura 12: Nube de puntos del estrés proyectando carga laboral

Si además generamos el plano de predicciones sobre el estrés en un rango de cargas laborales y salarios, podremos ver el modelo de predicción y compa-

arlo a la nube de puntos originales (conocidos).

```
1 import plotly.express as plotly
2
3 figure = plotly.scatter_3d(
4     estres_empleados,
5     x="CARGA",
6     y="SALARIO",
7     z="ESTRES",
8     color="ESTRES"
9 )
10
11 figure.update_traces(
12     marker=dict(
13         size=2,
14         #color="gray",
15         opacity=1
16     )
17 )
18
19 x1p = numpy.linspace(0, 20)
20 x2p = numpy.linspace(0, 30_000)
21
22 X1p, X2p = numpy.meshgrid(x1p, x2p)
23
24 yp = bias + X1p * b1 + X2p * b2
25
26 x1v = X1p.ravel()
27 x2v = X2p.ravel()
28
29 figure.add_trace(
30     go.Scatter3d(
31         x=x1v,
32         y=x2v,
33         z=yp.ravel(),
34         mode="markers",
35         marker=dict(
36             size=2,
37             color=yp.ravel(),
38             colorscale="inferno",
39             opacity=0.5
40         ),
41         name="Estres"
42     )
```

```

43 )
44
45 figure.show()

```

Con esto podríamos observar en la **Figura 13** el plano formado por (x_1, x_2) que son las características de carga laboral y salario, que responden al modelo $y = bias + x_1\beta_1 + x_2\beta_2$, o en general para la ecuación del hiperplano $a_1x_1 + a_2x_2 + \dots + a_kx_k + by = c$. Aquí ya se observa más claramente como

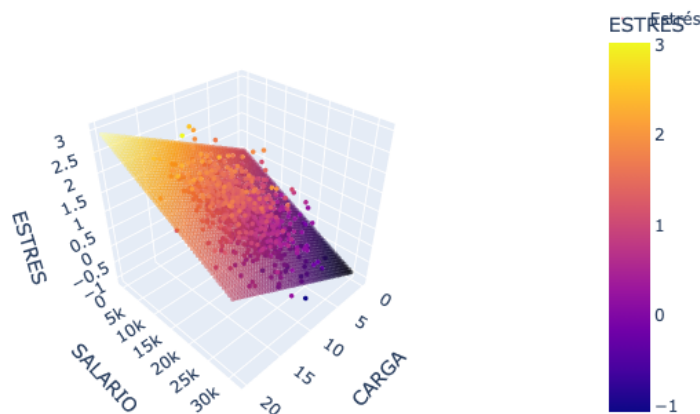


Figura 13: Hiperplano de predicción

el estrés tiene su máximo en la mayor carga laboral y el menor salario y su mínimo apuntando a la menor carga laboral y el mayor salario. También podemos ver cómo los puntos no están exactamente sobre el plano, sino que se alejan de él generando cierto error de predicción.

Las visualizaciones en 3D son complicadas de construir y requieren mayor programación y adaptaciones de una matriz *Mesh Grid* y la obtención de vectores coordenados *Ravel*. Y aunque con *plotly* tenemos gráficos bonitos e interactivos, optaremos mejor por producir gráficos 2D más interpretables, sobre todo para cuando tengamos más de dos características, en general $(x_1, x_2, \dots, x_k)$ en lugar de solo dos características (x_1, x_2) como en este ejemplo.

Con esto podemos crear un análisis estable en perspectiva de curvas de nivel

o proyecciones coordenadas utilizando el color como guía.

```
1 seaborn.scatterplot(  
2     x=x1v,  
3     y=x2v,  
4     hue=yp.ravel(),  
5     alpha=0.5,  
6     marker=".",  
7 )  
8 seaborn.scatterplot(  
9     x=X[:, 0],  
10    y=X[:, 1],  
11    hue=y,  
12    alpha=0.9,  
13 )
```

En la **Figura 14** se observan las predicciones del plano y la nube de puntos en 2D (como si se viera desde arriba). Observa que es muy fácil observar como

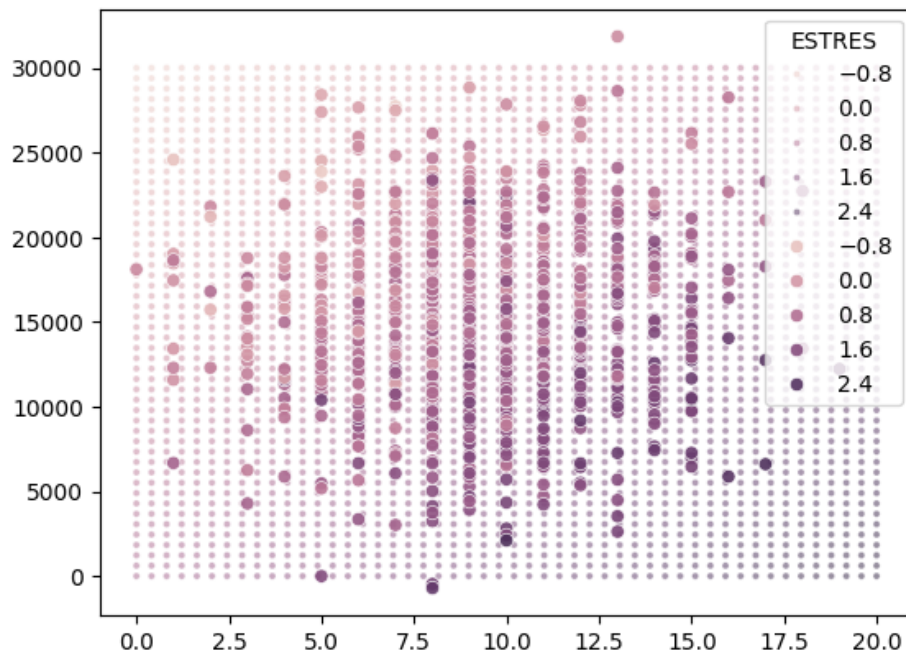


Figura 14: Plano de predicción y nube de puntos en 2D continuo

el estrés aumenta (color morado en este caso) cuando el salario disminuye o la carga laboral aumenta y en caso contrario alcanza un color rosa para un estrés bajo si el salario aumenta y la carga laboral disminuye.

También podemos simplificar una versión de contraste donde el estrés sea mayor a 1.

```

1 seaborn.scatterplot(
2     x=x1v,
3     y=x2v,
4     hue=yp.ravel() >= 1,
5     alpha=0.5,
6     marker="."
7 )
8 seaborn.scatterplot(
9     x=X[:, 0],
10    y=X[:, 1],
11    hue=y >= 1,
12    alpha=0.9,
13 )

```

En la **Figura 15** se observan las predicciones del plano y la nube de puntos en 2D en activación binaria. Observa que ahora el patrón es más notorio

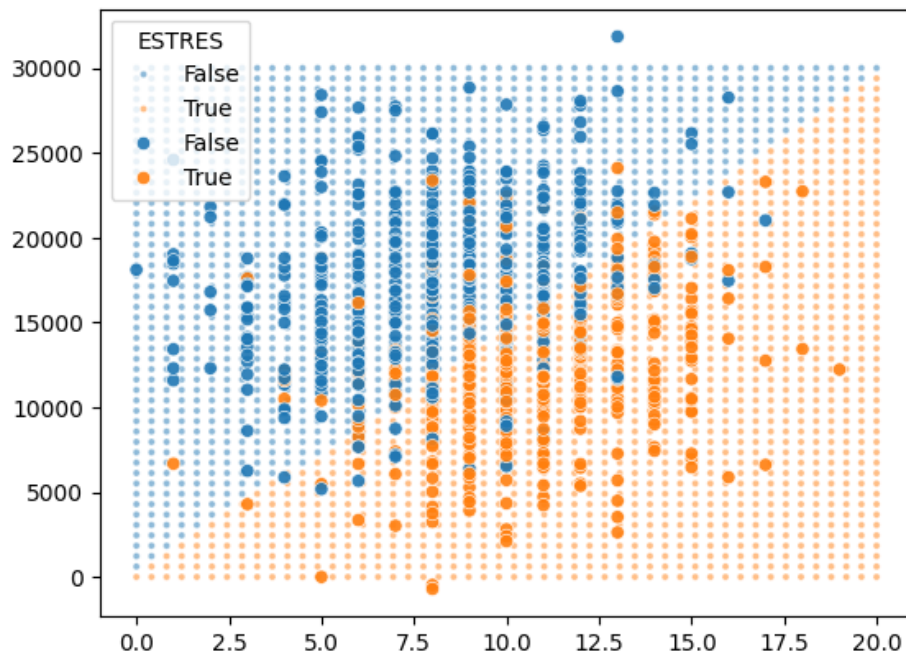


Figura 15: Plano de predicción y nube de puntos en 2D binaria

cuando el estrés supera el umbral de 1 o no lo hace. Y justamente el límite representará una carga laboral proporcional al salario.

Ahora podemos comparar el comportamiento del estrés en los datos de prueba y_{test} y la predicción del estrés en los datos de prueba $\hat{y}_{test}$.

```

1 yp_test = bias + X_test[:, 0] * b1 + X_test[:, 1] * b2
2
3 figure, axis = pyplot.subplots(1, 2, figsize=(20, 6))
4
5 seaborn.scatterplot(
6     x=X_test[:, 0],
7     y=X_test[:, 1],
8     hue=y_test,
9     alpha=0.9,
10    ax=axis[0]
11 )
12 seaborn.scatterplot(
13     x=X_test[:, 0],
14     y=X_test[:, 1],
15     hue=yp_test,
16     alpha=0.9,
17     ax=axis[1]
18 )

```

En la **Figura 16** se compara el estrés de los datos de prueba y_{test} y las predicciones para los datos de prueba $\hat{y}_{test}$.

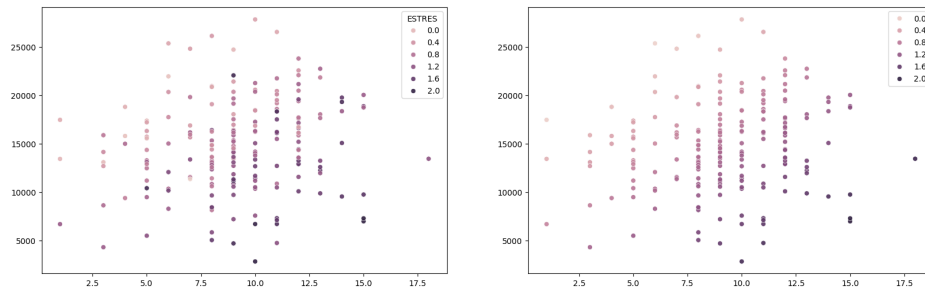


Figura 16: Izquierda: datos de prueba y_{test} , Derecha: $\hat{y}_{test}$

Las predicciones colorean prácticamente igual, esto sugiere que nuestro modelo de regresión lineal múltiple es muy bueno prediciendo.

Si visualizamos el error cuadrático o diferencias al cuadrado entre las respuestas de prueba y las predicciones $(y_{test} - \hat{y}_{test})^2$, tendremos una gráfica que se colorea cuando el error cuadrático aumenta, dejando en un color más visible dónde las predicciones fallan.

```

1  seaborn.scatterplot(
2      x=X_test[:, 0],
3      y=X_test[:, 1],
4      alpha=0.8,
5      color="gray",
6      marker="."
7  )
8  seaborn.scatterplot(
9      x=X_test[:, 0],
10     y=X_test[:, 1],
11     hue=(y_test - yp_test) ** 2,
12     alpha=0.8,
13     palette="Reds",
14     marker="."
15 )

```

En la **Figura 17** se visualiza el error cuadrático entre las respuestas y predicciones $(y_{test} - \hat{y}_{test})^2$.

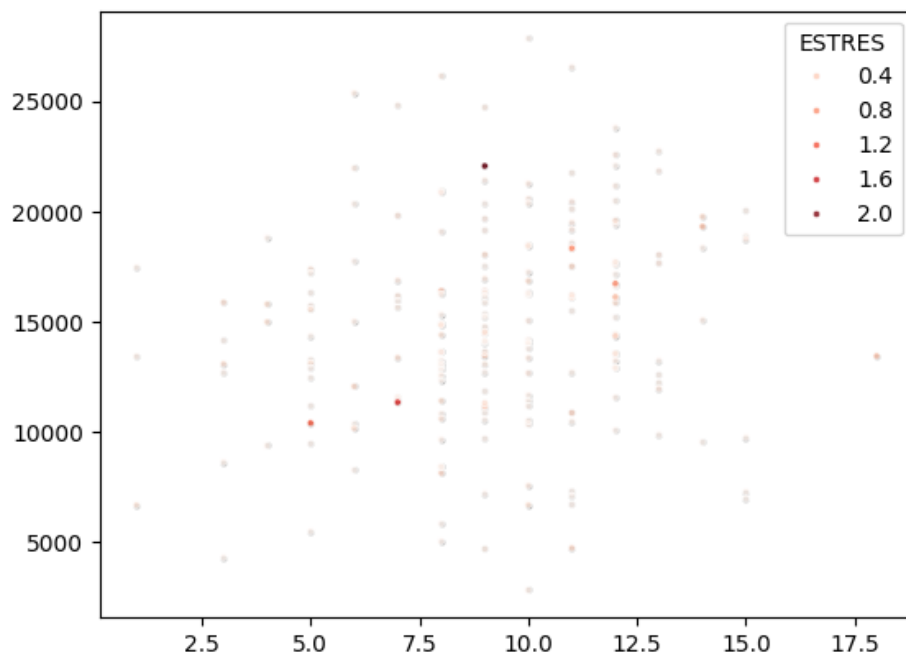


Figura 17: Error cuadrático en los datos de prueba

Observamos que el error colore prácticamente muy pocos puntos con diferencias grandes y la mayoría no superan el 0,4 de error cuadrático.

Este error se ve mejor si enfocamos más la gráfica en un histograma ponderado.

```
1 seaborn.histplot(  
2     x=X_test[:, 0],  
3     y=X_test[:, 1],  
4     weights=(y_test - yp_test) ** 2,  
5     bins=30,  
6     cbar=True,  
7     cmap="Blues"  
8 )
```

En la **Figura 18** se visualiza el histograma ponderado del error cuadrático.

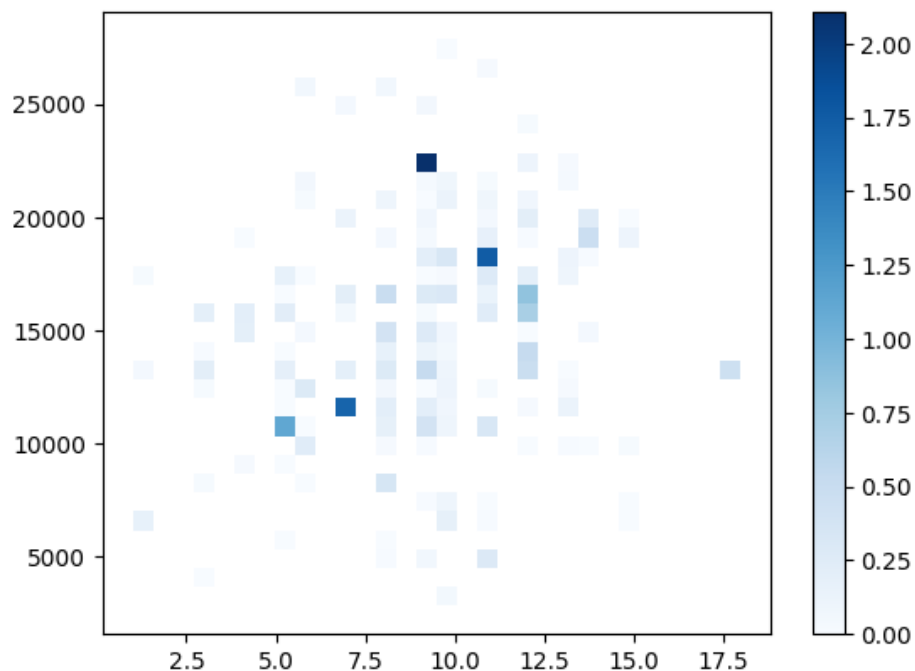


Figura 18: Error cuadrático ponderado en histograma

Observamos que son menos de 10 puntos los que tienen un error cuadrático superior a 0,5.

O mejor aún podemos ponderar los puntos para visualizarlo en forma de puntos pesados.

```
1 seaborn.scatterplot(  
2     x=X_test[:, 0],
```



```

3     y=X_test[:, 1],
4     alpha=0.2,
5     color="gray",
6     marker="."
7 )
8 seaborn.scatterplot(
9     x=X_test[:, 0],
10    y=X_test[:, 1],
11    size=(y_test - yp_test) ** 2,
12    hue=(y_test - yp_test) ** 2,
13    palette="Blues",
14    sizes=(0, 100),
15    #legend=False,
16 )

```

En la **Figura 19** se visualizan los puntos ponderado del error cuadrático.

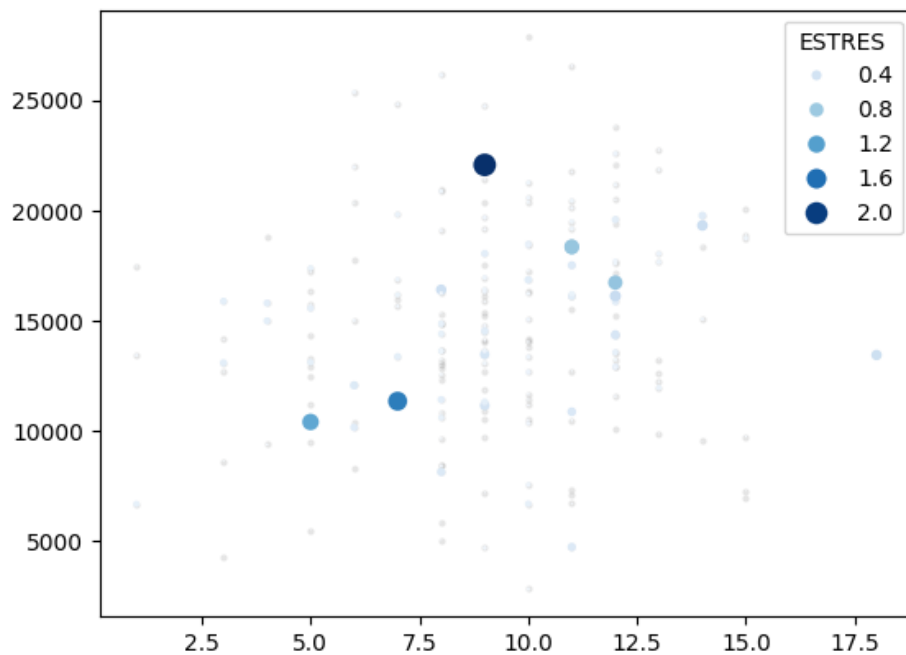


Figura 19: Error cuadrático ponderado en puntos

Aquí ya es muchísimo más fácil ver qué puntos superar un error cuadrático de 1.

Otra idea final de visualización sería determinar qué datos superan una banda

de error y visualizarlo en 3D.

```
1 import plotly.express as plotly
2
3 yp = bias + X[:, 0] * b1 + X[:, 1] * b2
4
5 figure = plotly.scatter_3d(
6     x=X[:, 0],
7     y=X[:, 1],
8     z=y,
9     color=(y - yp).abs() >= 0.3,
10    color_continuous_scale="inferno"
11 )
12
13 figure.update_traces(
14     marker=dict(
15         size=2,
16         #color="gray",
17         opacity=1
18     )
19 )
20
21 figure.show()
```

En la **Figura 20** se visualizan los puntos que superan 0,3 del error absoluto.

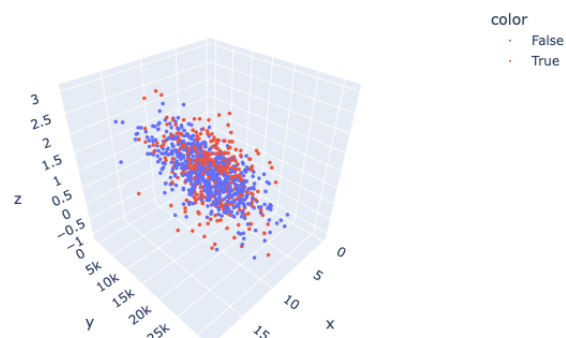


Figura 20: Nube de puntos con banda de error absoluta de 0,3

En esta visualización podemos observar que los puntos que tienen un error de predicción del estrés mayor a 0,3 en términos absolutos, se colorean en rojo, representando una masa pequeña comparada a todos los puntos azules que representan datos cuya predicción tiene un error absoluto menor o igual 0,3.

Esta banda es importante para determinar qué porcentaje de los datos queda correctamente predicho bajo cierto margen de error admitido. Por ejemplo, si un estrés cercano a 1 se considera entre (0,7, 1,3), entonces, las predicciones concentrarían el 68 % de los datos (interesante su similitud con la desviación estándar).

Finalmente, podemos crear las métricas que nos digan qué tan bien se ajustan los datos de prueba al modelo de predicción:

```

1 print("SSE:", ((y_test - yp_test) ** 2).sum())
2 print("MSE:", ((y_test - yp_test) ** 2).mean())
3 print("-" * 40)
4 print("MIN", (y - yp).abs().min())
5 print("MAE:", (y_test - yp_test).abs().mean())
6 print("MAX", (y - yp).abs().max())
7 print("RMSE:", ((y_test - yp_test) ** 2).mean() ** 0.5)
8 print("(MAE + RMSE)/2:", ((y_test - yp_test).abs().mean()
9      + ((y_test - yp_test) ** 2).mean() ** 0.5) / 2)
10 print("RMSE/MAE:", ((y_test - yp_test) ** 2).mean() **
11      0.5 / (y_test - yp_test).abs().mean())
12 print("-" * 40)
13 print("< 0.3:", ((y - yp).abs() <= 0.3).mean())
14 print("< 0.6:", ((y - yp).abs() <= 0.6).mean())
15 print("< 0.9:", ((y - yp).abs() <= 0.9).mean())
16 print("< 1.2:", ((y - yp).abs() <= 1.2).mean())
17 print("< 0.6:", ((y - yp).abs() <= 0.6).mean())
18 print("< 0.9:", ((y - yp).abs() <= 0.9).mean())
19 print("< 1.2:", ((y - yp).abs() <= 1.2).mean())

```

SSE: 23.107552070016425

MSE: 0.11553776035008212

MIN 0.000397748014032695

MAE: 0.25817661701718775

MAX 1.5194472929406821

RMSE: 0.339908458779834

(MAE + RMSE)/2: 0.2990425378985109

RMSE/MAE: 1.3165733702258755

< 0.3: 0.678
< 0.6: 0.911
< 0.9: 0.978
< 1.2: 0.992
< 0.6: 0.911
< 0.9: 0.978
< 1.2: 0.992

donde, las métricas representan:

- **SSE** (*Sum Square Error / Suma del Error Cuadrático*) - Representa cuánto error cuadrático se acumula entre la respuesta esperada y la de predicción, formando una masa de error.
- **MSE** (*Mean Square Error / Promedio del Error Cuadrático*) - Representa cuánto error cuadrático hay en promedio, aproximando algo similar a la varianza de error entre la respuesta esperada y la de predicción.
- **RMSE** (*Root Mean Square Error / Raíz cuadrada del Promedio del Error Cuadrático*) - Representa cuánto error radial hay en promedio, aproximando algo similar a la desviación de error entre la respuesta esperada y la de predicción.
- **MAE** (*Mean Absolute Error / Promedio del Error Absoluto*) - Representa cuánto error absoluto hay en promedio, aproximando algo similar a una franja de error entre la respuesta esperada y la de predicción.
- $< \lambda$ - Representa la proporción de datos que quedan atrapados en un error máximo λ ($\varepsilon < \lambda$). Por ejemplo, para $\lambda = 0,3$, la proporción de datos con un error máximo $\varepsilon < 0,3$ es de 68%. Esto se puede interpretar como la proporción de datos correctamente predichos bajo una tolerancia λ de error.

Para cerrar este estudio, mostraremos como resultado final la gráfica de cómo aumenta la proporción o captura de datos (cobertura) cuando la tolerancia aumenta.

```
1 umbrales = numpy.linspace(0, 1.5, 100)
2
3 cobertura = [
4     ((y_test - yp_test).abs() <= t).mean()
```

```

5     for t in umbrales
6 ]
7
8 pyplot.plot(umbrales, cobertura)
9 pyplot.xlabel("Tolerancia de error")
10 pyplot.ylabel("Cobertura")

```

En la **Figura 21** se visualiza la cobertura de predicción o captura de la proporción de los datos bajo una tolerancia de error.

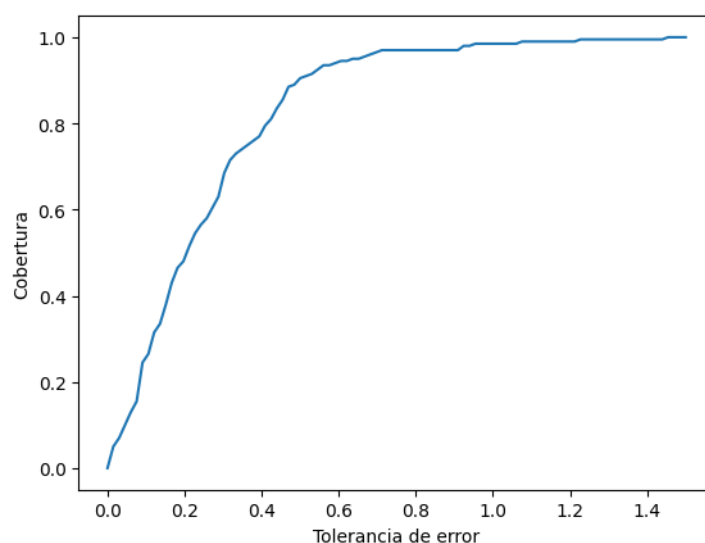


Figura 21: Curva de cobertura en la proporción de datos correctamente predichos bajo la tolerancia de error $\lambda \in (0, 1,5)$

En un modelo ideal, esperaríamos que la curva suba rápidamente, representando un error bajo para lograr cubrir todos los datos con poca tolerancia. En este caso vemos como hasta cerca de una tolerancia de 0,5 se comienzan a cubrir el 85 % de los datos, lo que significa que una predicción con 85 % de veracidad tendría un error absoluto de 0,5 en la predicción, por ejemplo, si la predicción es 1,2, tendríamos un rango entre (0,7, 1,7) que sugeriría que el valor se encuentra ahí, salvo que sea parte del 15 % de los valores que no están dentro de la banda del 0,5 de error.

Bibliografía

- Géron, A. (2023). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (3.^a ed.). O'Reilly Media.
- Chollet, F. (2021). Deep Learning with Python (2.^a ed.). Manning Publications.
- Raschka, S., & Mirjalili, V. (2022). Python Machine Learning: Machine Learning and Deep Learning with Python, scikit-learn, and TensorFlow (3.^a ed.). Packt Publishing.
- Müller, A. C., & Guido, S. (2016). Introduction to Machine Learning with Python: A Guide for Data Scientists. O'Reilly Media.
- Grus, J. (2019). Data Science from Scratch: First Principles with Python (2.^a ed.). O'Reilly Media.
- Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4.^a ed.). Pearson.